LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE CR002
PERTEMUAN 5

Penyimpanan Lokal (SharedPreferences dan SQLite)

Disusun Oleh:

Arum Syafitri Puspita Pratiwi (20240801010)

Dosen Pengampu:

JEFRY SUNUPURWA ASRI , S.Kom., M.Kom

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

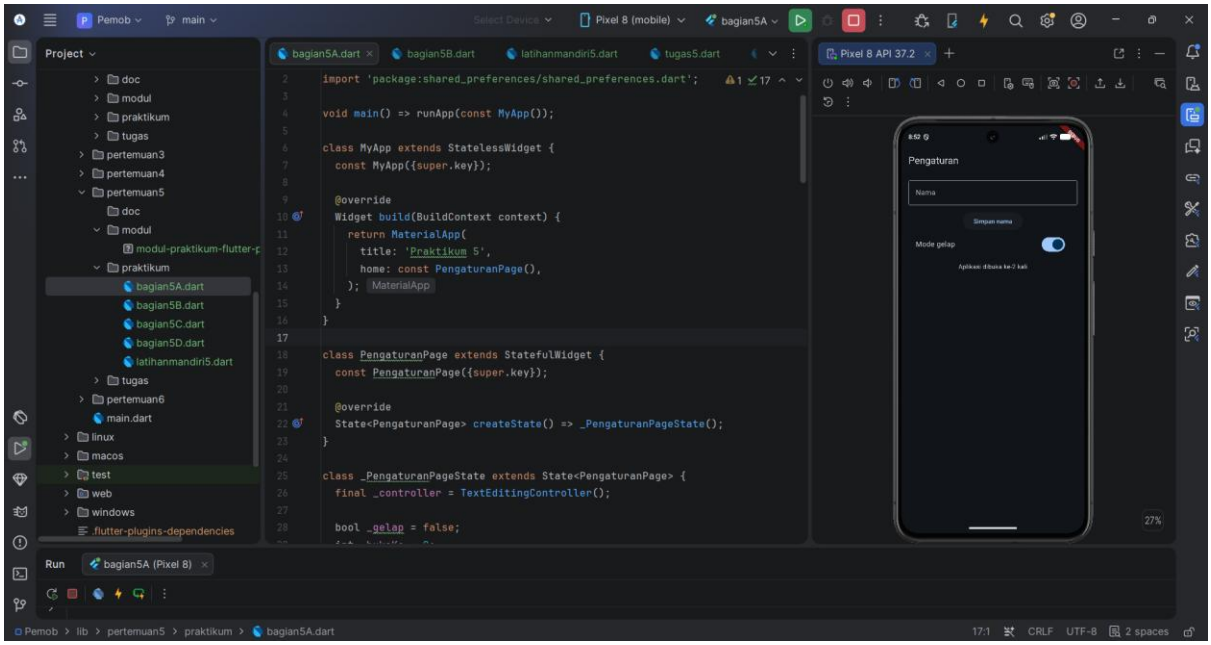
UNIVERSITAS ESA UNGGUL

2026

Bagian A: SharedPreferences (Pengaturan Sederhana)

```
C:\Pemob>flutter pub add shared_preferences
Resolving dependencies...
Downloading packages... (25.5s)
  cupertino_icons 1.0.9 (2.0.0 available)
+ ffi 2.2.0
+ file 7.0.1
+ flutter_web_plugins 0.0.0 from sdk flutter
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
+ path_provider_linux 2.2.2
+ path_provider_platform_interface 2.1.3
+ path_provider_windows 2.3.0
+ platform 3.2.0
+ plugin_platform_interface 2.1.8
+ shared_preferences 2.5.5
+ shared_preferences_android 2.4.28
+ shared_preferences_foundation 2.5.7
+ shared_preferences_linux 2.4.1
+ shared_preferences_platform_interface 2.4.2
+ shared_preferences_web 2.4.3
+ shared_preferences_windows 2.4.1
  test_api 0.7.12 (0.7.14 available)
  vector_math 2.4.0 (2.4.3 available)
+ xdg_directories 1.1.0
Changed 16 dependencies!
5 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
```

Praktikum bagian 5A ini membahas implementasi penyimpanan lokal menggunakan paket shared_preferences untuk menyimpan data sederhana seperti nama pengguna, pengaturan mode gelap, serta penghitung jumlah berapa kali aplikasi dibuka.

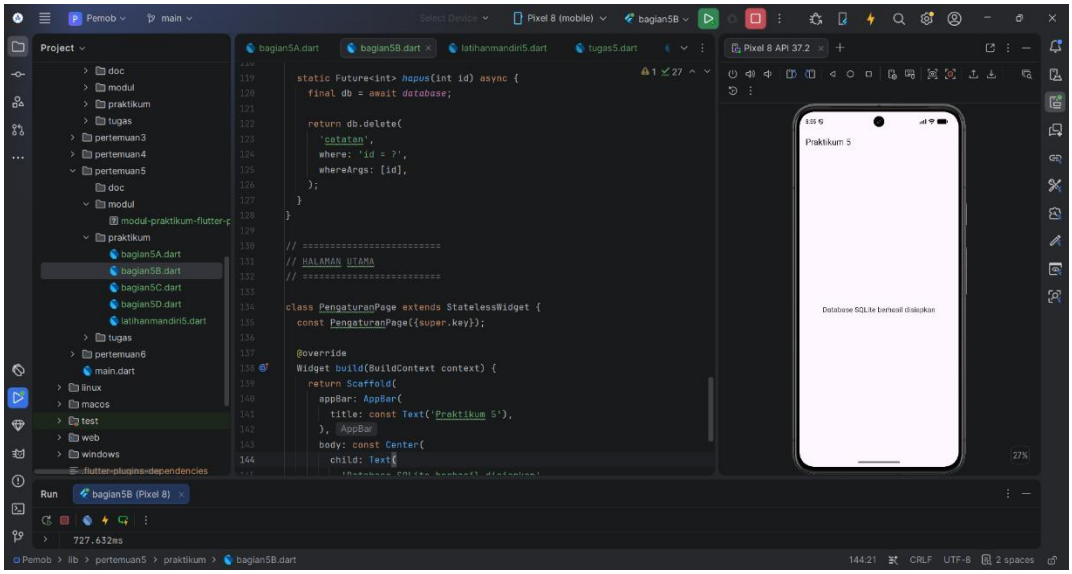


Tampilan antarmuka dan emulator pada bagian 5A ini menunjukkan halaman pengaturan aplikasi yang berhasil dijalankan dengan fitur tema mode gelap aktif serta informasi penghitung jumlah pembukaan aplikasi.

Bagian B: Menyiapkan SQLite

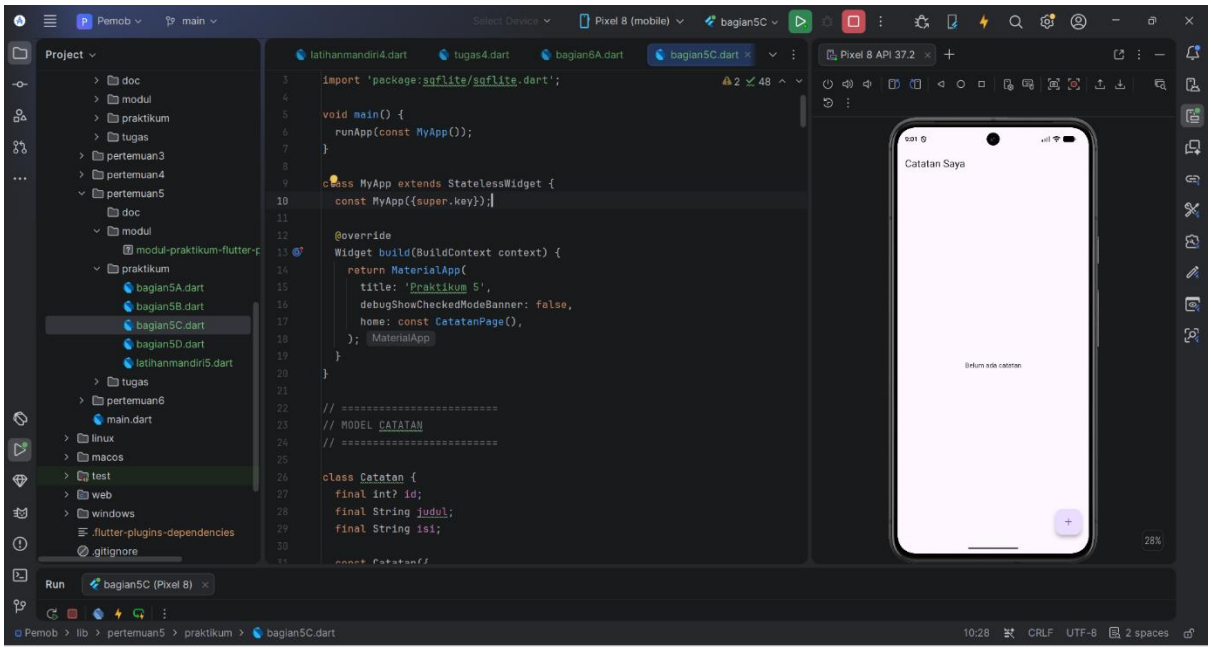
```
C:\Pemob>flutter pub add sqflite path
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  path 1.9.1 (from transitive dependency to direct dependency)
+ sqflite 2.4.4
+ sqflite_android 2.4.4
+ sqflite_common 2.5.13
+ sqflite_darwin 2.4.4
+ sqflite_platform_interface 2.4.2
+ synchronized 3.4.2
  test_api 0.7.12 (0.7.14 available)
  vector_math 2.4.0 (2.4.3 available)
Changed 7 dependencies!
5 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
```

Gambar tersebut menunjukkan proses instalasi paket pustaka tambahan pada terminal menggunakan perintah flutter pub add sqflite path. Perintah ini berfungsi untuk mengunduh dan menambahkan dependensi sqflite yang digunakan untuk mengelola database SQLite serta paket path untuk mengatur direktori penyimpanan database lokal di perangkat.



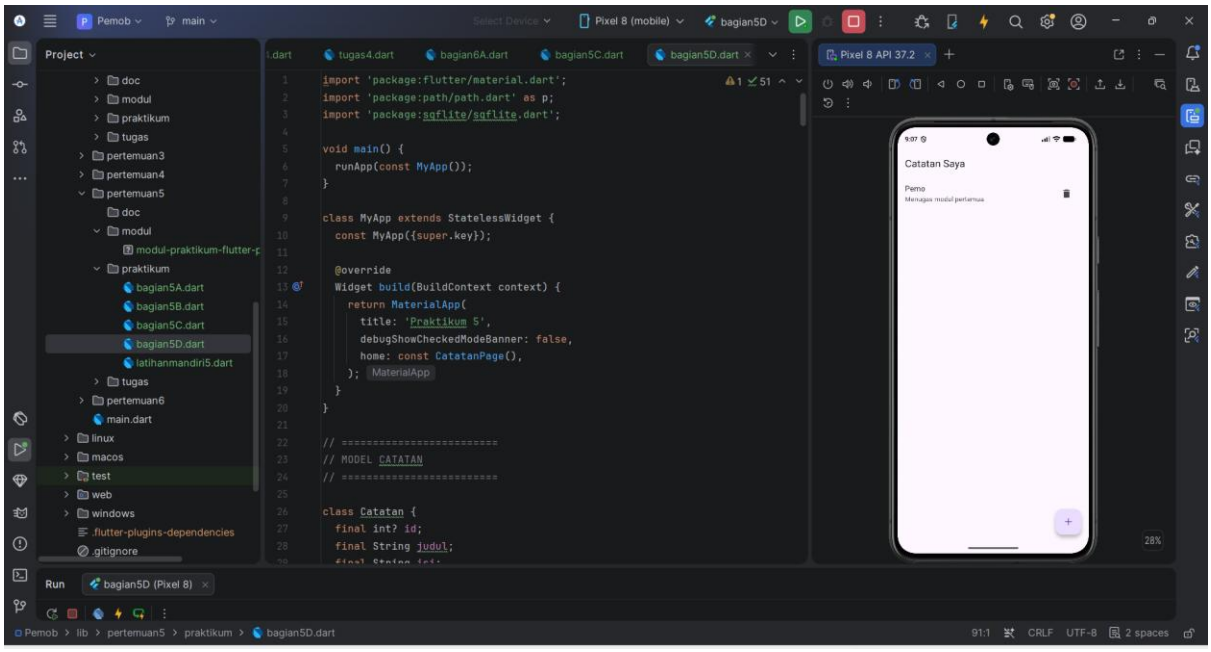
Tampilan antarmuka dan emulator pada bagian 5B ini menunjukkan implementasi kode program yang berhasil dijalankan, menampilkan halaman utama aplikasi dengan teks indikator bahwa struktur model data dan database SQLite telah berhasil disiapkan pada sistem.

Bagian C: Halaman Daftar Catatan (Read dan Delete)



Praktikum bagian 5C ini berfokus pada implementasi halaman daftar catatan (CatatanPage) yang menerapkan operasi pembacaan data (Read) dari database SQLite serta penghapusan data (Delete) menggunakan komponen ListView.builder dan FutureBuilder. Berdasarkan hasil pengujian pada emulator, aplikasi berhasil menampilkan antarmuka utama dengan judul "Catatan Saya", indikator informasi kosong karena belum ada data catatan, serta tombol aksi mengambang (*floating action button*) bertuliskan ikon tambah untuk bernavigasi ke halaman form pengelolaan catatan.

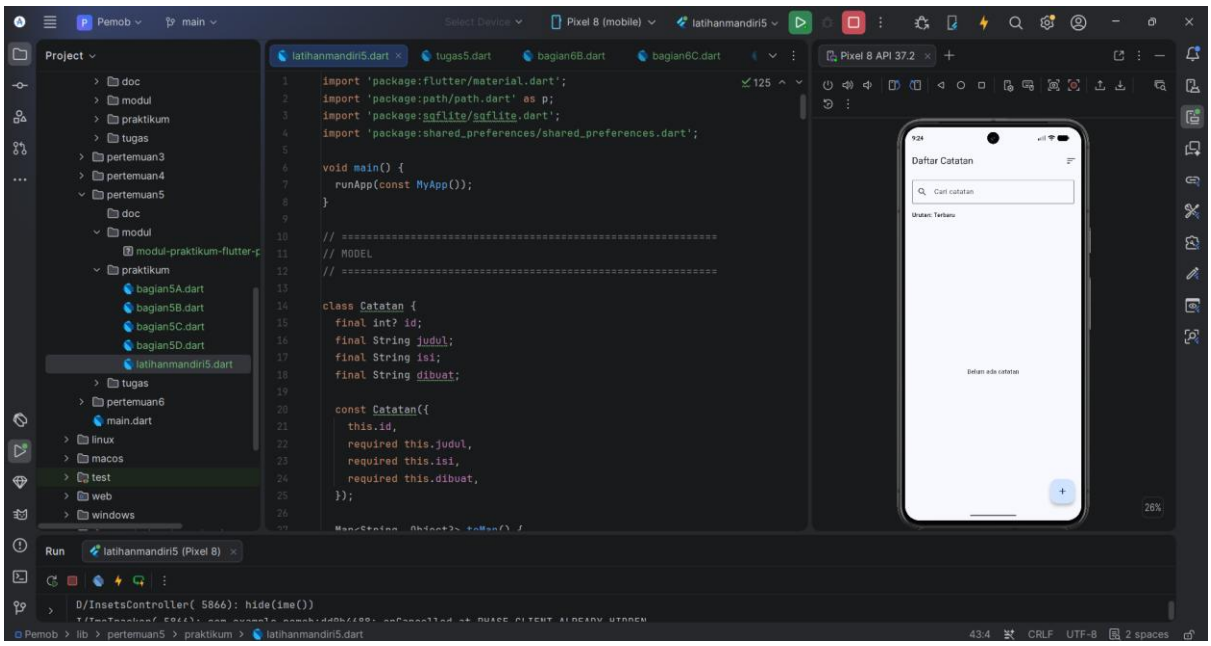
Bagian D: Form Tambah dan Ubah Catatan (Create dan Update)



Praktikum bagian 5D ini mengimplementasikan halaman form dinamis (FormCatatanPage) yang digunakan secara fleksibel untuk dua fungsi sekaligus, yaitu menambah catatan baru

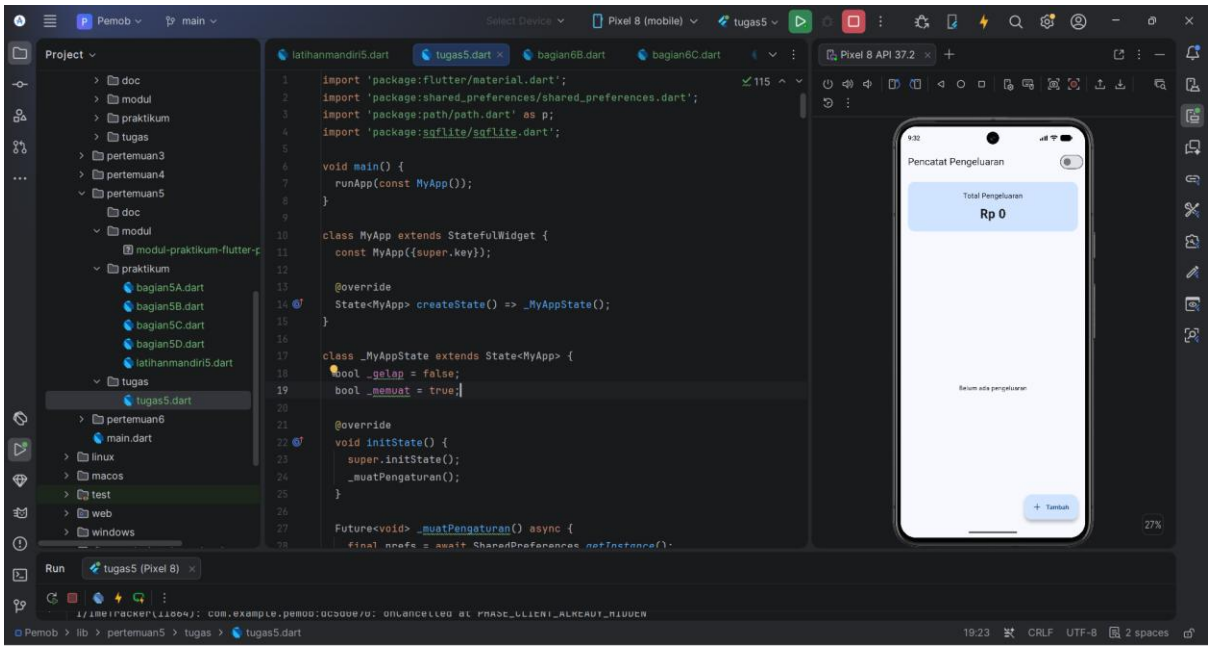
apabila data kosong atau memperbarui data catatan yang sudah ada. Berdasarkan hasil pengujian pada emulator, form dilengkapi dengan validasi input teks serta tombol simpan yang berhasil memproses operasi *Create* dan *Update* ke dalam database SQLite, sehingga data catatan dapat tampil secara dinamis pada daftar utama dan tetap persisten setelah aplikasi ditutup.

Latihan Mandiri



Di bagian Latihan Mandiri ini, kita menambahkan beberapa fitur tambahan biar aplikasinya makin lengkap. Mulai dari bikin dialog konfirmasi pas mau hapus catatan, nambahin kolom pencarian, naikin versi database buat migrasi kolom baru, sampai nyimpen pilihan urutan tampilan pakai shared_preferences. Pas dijalankan di emulator, aplikasinya udah tampil rapi dengan kolom pencarian di bagian atas, info status urutan data, serta daftar catatan interaktif lengkap dengan tombol hapus dan menu pengurutannya

Tugas



Di bagian Tugas utama ini, kita diminta bikin aplikasi "Pencatat Pengeluaran" yang fiturnya lengkap banget sesuai ketentuan modul. Mulai dari bikin tabel khusus pengeluaran dengan data id, nama, jumlah, kategori, dan tanggal; operasi CRUD lengkap dengan form validasi (nama wajib diisi, jumlah angka di atas 0, dan kategori pakai dropdown); nampilin total pengeluaran di halaman utama; nyimpen pengaturan (kayak mode gelap) pakai shared_preferences; sampai mastikan datanya aman tetap kesimpen pas aplikasi ditutup dan dibuka ulang. Pas dites di emulator, tampilannya udah sukses jalan dengan kartu total pengeluaran di atas, tombol tema, list kosong atau data pengeluaran, serta tombol tambah di pojok bawah.

Pertanyaan Refleksi

1. Mengapa shared_preferences kurang cocok untuk menyimpan ratusan catatan? Apa pilihan yang lebih baik?

   Jawab: Karena shared_preferences dirancang khusus untuk menyimpan data kecil berformat *key-value* (seperti pengaturan tema, nama, atau status login), bukan untuk data yang banyak dan terstruktur. Kalau dipaksa buat nyimpen ratusan catatan, performanya bakal lambat dan tidak efisien. Pilihan yang jauh lebih baik adalah menggunakan database lokal seperti **SQLite** (sqflite).

2. Mengapa query memakai placeholder? dan whereArgs, bukan menyambung teks langsung?

   Jawab: Biar aplikasi aman dari celah keamanan **SQL Injection**, di mana pengguna bisa menyisipkan perintah SQL berbahaya lewat teks input. Dengan pakai tanda tanya (?) sebagai *placeholder* dan whereArgs, input dari pengguna akan dibaca secara aman murni sebagai nilai data, bukan sebagai perintah program.

3. Mengapa setelah kembali dari halaman form, daftar perlu dimuat ulang? Bagaimana caranya pada kode ini?

   Jawab: Daftar perlu dimuat ulang supaya tampilan di layar utama langsung memperbarui data terbaru setelah kita nambah, edit, atau hapus catatan. Pada kode tersebut, caranya dipanggil lewat fungsi _muat() tepat setelah selesai dari navigasi form (Navigator.push) atau setelah kembali dari halaman form.

4. Apa fungsi onCreate dan onUpgrade, dan kapan masing-masing dijalankan?

   Jawab:

   - onCreate: Berfungsi untuk menjalankan perintah pembuatan tabel baru di database dan hanya dijalankan sekali ketika database tersebut pertama kali dibuat.
   - onUpgrade: Berfungsi untuk memperbarui atau migrasi skema database yang sudah ada (misalnya nambah kolom baru pakai ALTER TABLE) dan dijalankan otomatis saat nomor versi database dinaikkan dari versi sebelumnya.

Kesimpulan

Secara keseluruhan, praktikum pertemuan kelima ini membahas cara menyimpan data secara lokal di aplikasi Flutter agar tidak hilang saat aplikasi ditutup. Kita mempelajari penggunaan shared_preferences untuk pengaturan sederhana yang berformat *key-value* serta database SQLite menggunakan sqflite untuk menyimpan data terstruktur seperti daftar catatan dan pencatat pengeluaran secara lengkap melalui operasi CRUD.